

SCJP 1.6 (CX-310-065 , CX-310-066)

Subject: Mock Questions Set-1

Total Questions : 60

Prepared by : <http://www.javacertifications.net>

SCJP 6.0: Full Length Mock Exam Set-1(60 Questions)

Questions Question - 1

What is the output ?

```
public interface TestInf {
    int i =10;
}

public class Test {
    public static void main(String... args) {
        TestInf.i=12;
        System.out.println(TestInf.i);
    }
}
```

- 1.Compile with error
- 2.10
- 3.12
- 4.None of the above

Explanation :

A is the correct answer.

All the variables declared in interface is Implicitly static and final , so can't change the value.

Question - 2

Fill all the gaps

```
try{
    File f = new File("a.txt");
    }catch( e){

    }catch( io){

    }
}
```

Use the following fragments zero or many times

Exception

IOException

Question - 3

```
try{
    File f = new File("a.txt");
    }catch(Exception e){

    }catch(IOException io){

    }
}
Is this code create new file name a.txt ?
```

- 1.True
- 2.False
- 3.Compilation Error
- 4.None

Explanation :

C is the correct answer.

IOException is unreachable to compiler because all exception is going to catch by Exception block.

Question - 4

What is output of the below code ?

```
public class Test {

    public static void main(String... args) throws Exception {
        Queue<String> lst = new PriorityQueue<String>();
        lst.add("one");
        lst.add("two");
        System.out.println(lst.poll() + " " + lst.poll() + " "
+lst.poll());
    }
}
```

- 1.one two null]
- 2.59
- 3.Runtime Exceptions
- 4.None of the above

Explanation :

B is the correct answer.

Manual conversion between primitive types (such as an int) and wrapper classes (such as Integer) is necessary when adding a primitive data type to a collection in jdk1.4 but The new autoboxing/unboxing feature eliminates this manual conversion in jdk 1.5

Question - 7

Fill in the gap:

```
public class Test {  
  
    public static void main(String[] args) {  
        String[] words = new String[] {"aaa", "bbb", "ccc", "aaa"};  
        Map<String, Integer> m = new TreeMap<String, Integer>();  
        for (String word : words) {  
            freq = m.get(word);  
            m.put(word, freq == null ? 1 : freq + 1);  
        }  
        System.out.println(m);  
    }  
}
```

Use the following fragments zero or many times

String

Integer

Boolean

Float

Question - 8

Which of the following statements is correct with regard to threads.

- 1."Runnable" is a keyword within the Java language, used to identify classes which can exist as separate threads.
- 2."Runnable" is an interface, which classes can implement when they wish to execute as a separate thread.
- 3."Runnable" is a class, which classes can extend when they wish to execute as a separate thread.
- 4.None of the above.

Explanation :

B is the correct answer.

Runnable is an abstract interface.

Question - 9

Fill in all the gaps using the available fragments:

```

interface Bashable { };

interface Test Bashable {
    int smiteForce = 11;
    void beginSmiting();
}

```

Use the following fragments zero or many times

implements

abstract

strictfp

extends

native

private

static

Question - 10

Read this piece of code carefully

```

if("String".trim() == "String")
    System.out.println("Equal");
else
    System.out.println("Not Equal");

```

- 1.the code will compile an print "Equal".
- 2.the code will compile an print "Not Equal".
- 3.Not Compile
- 4.None

Explanation :

A is the correct answer.

trim() Returns a copy of the string, with leading and trailing whitespace omitted. This program compares address of "String" get from the String reference pool. Every new String checks in the String Reference pool, if already present then return the address of the existing String otherwise create new one and add to String reference pool.

Question - 11

```

class C {
public static void main(String[] args) {
    System.out.println(4+5+"String");
}}

```

- 1.prints 9string
- 2.prints 45string
- 3.compile time error
- 4.Runtime exception

Explanation :

A is the correct answer.

arguments to the method are evaluated from left to right so $4+5+\text{"string"} \Rightarrow 9+\text{string} \Rightarrow 9\text{string}$

Question - 12

What is output of the below code ?

```
class C {  
    public static void main ( String ka [ ] ) {  
        while ( false ) {  
            System.out.println ( "Value" ) ;  
        }  
    }  
}
```

- 1.compile time error
- 2.prints Value infinitely
- 3.Runtime Exception
- 4.None of the above

Explanation :

A is the correct answer.

while (false) is Unreachable Code.

Question - 13

What is output of the below code ?

```
interface I{  
    final class C1 { //1  
        static int i=9; //2  
    }  
}  
class C2 implements I{  
    public static void main(String a[]){  
        System.out.println(I.C1.i); ///3  
    }  
}
```

- 1.compile time error at line 1
- 2.compile time error at line 2
- 3.compile time error at line 3
- 4.prints 9

Explanation :

D is the correct answer.

interfaces classes are by default static,final. so,no compile time errors are generated

Question - 14

The constructor for the Math class is private, so it cannot be instantiated

- 1.true
- 2.false
- 3.none of the above
- 4.none of the above

Explanation :

A is the correct answer.

Private is not visible to outside of class. So you can't create instance of Math class outside of Math class.

Question - 15

What is output of the below code ?

```
class C {  
    static void m1(Object x) {  
        System.out.print("Object");  
    }  
    static void m1(String x) {  
        System.out.print("String");  
    }  
    public static void main(String[] args) {  
        m1(null);  
    }  
}
```

- 1.Prints Object
- 2.Prints String
- 3.compiletime error
- 4.None of the above

Explanation :

B is the correct answer.

The more specific of the two, m(String x), is chosen

Question - 16

What is output of the below code ?

```
import java.util.NavigableMap;
```

```

import java.util.concurrent.ConcurrentSkipListMap;

public class Test {
    public static void main(String... args) {

        NavigableMap <Integer, String>navMap = new
        ConcurrentSkipListMap<Integer, String>();

        navMap.put(4, "April");
        navMap.put(5, "May");
        navMap.put(6, "June");
        navMap.put(1, "January");
        navMap.put(2, "February");
        navMap.put(3, "March");

        navMap.pollFirstEntry();
        navMap.pollLastEntry();
        navMap.pollFirstEntry();
        System.out.println(navMap.size());

    }
}

```

1.Compile error : No method name like pollFirstEntry() or pollLastEntry()

2.3

3.6

4.None of the above

Explanation :

B is the correct answer.

< pollFirstEntry() Removes and returns a key-value mapping associated with the least key in this map, or null if the map is empty.

H pollLastEntry() Removes and returns a key-value mapping associated with the greatest key in this map, or null if the map is empty.

Question - 17

What is output of the below code ?

```

class c1
{
    public void m1()
    {
        System.out.println("m1 method in C1 class");
    }
}
class c2
{
    public c1 m1()
    {
        return new c1(){
            public void m1()
            {
                System.out.println("m1 mehod in anonymous class");
            }
        }
    }
}

```

```

    });}
public static void main(String a[])
{

    c1 ob1 =new c2().m1();
        ob1.m1();
}}

```

- 1.prints m1 method in C1 class
- 2.prints m1 method in anonymous class
- 3.compile time error
- 4.Runtime error

Explanation :

B is the correct answer.

the anonymous class overrides the m1 method in c1.so according to the dynamic dispatch the m1 method in the anonymous is called

Question - 18

What is output of the below code ?

```

class C {

    public static void main ( String ka [ ] ) {

        Thread t = Thread . currentThread ( ) ;
            t . setPriority ( - 1 ) ;
            System . out . println ( " Done ! " ) ;
    }
};

```

- 1.compile time error
- 2.Runtime Exception
- 3.The code compiles and runs fine
- 4.None of the above

Explanation :

B is the correct answer.

Code for setPriority is : if (newPriority greater then MAX_PRIORITY || newPriority less then MIN_PRIORITY) { throw new IllegalArgumentException(); } where MAX_PRIORITY = 10 and MIN_PRIORITY =1 , so setPriority should not be less then 1 and greater then 10.

Question - 19

```

abstract class A {}    // 1
transient class B {}   // 2

```



```
private class C {}    // 3
static class D {}    // 4
```

Which of these declarations will not produce a compile-time error?

- 1.1
- 2.2
- 3.3
- 4.4

Explanation :

A is the correct answer.

The modifiers, private and static, can be applied to a nested class, but can not be applied to a class that is not nested transient is allowed only for variables.

Question - 20

Which of the following is correct?

- 1.String temp [] = new String {"j" "a" "z"};
- 2.String temp [] = { "j " " b" "c"};
- 3.String temp = {"a", "b", "c"};
- 4.String temp [] = {"a", "b", "c"};

Explanation :

D is the correct answer.

String temp [] = {"a", "b", "c"}; where temp[] is String array.

Question - 21

What is the output of the below code ?

```
abstract class vehicle{
abstract public void speed();
}
class car extends vehicle{
public static void main (String args[]) {
vehicle ob1;
ob1=new car();    //1

}}
```

- 1.compiletime error at line 1
- 2.forces the class car to be declared as abstract
- 3.Runtime Exception
- 4.None of the above

Explanation :

B is the correct answer.

Abstract method should be overridden otherwise Subclass should be abstract.

Question - 22

Which statement is true about java.io.Console class ?

1. you can't extend Console class because it is final
2. Console class is not final]
3. Console implements Flushable
4. None of the above

Explanation :

A and C is the correct answer.

public final class Console implements Flushable {}. you can't extend Console class because it is final.

Question - 23

Fill the gap to get output aa ab ac

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class Test {
    public static void main(String... args) {

        List<String> a = new ArrayList<String>();
        a.add("aa"); a.add("ac"); a.add("ab");

        // insert code

        for(String s1: a) System.out.println(s1);
    }
}
```

Use each fragment at most once

Collections.sort(a);

Collections.reverse(a);

```
Collections.sort(a);
```

Question - 24

What is output of the below code ?

```
abstract class C1{  
public void m1(){           //1  
}  
}  
abstract class C2{  
public void m2(){           //2  
}  
}
```

- 1.compile time error at line1
- 2.compile time error at line2
- 3.The code compiles fine
- 4.None of the above

Explanation :

C is the correct answer.

since the class C2 is abstract it can contain abstract methods

Question - 25

What is the output of the below code ?

```
public class NameBean {  
private String str;  
  
NameBean(String str ){  
this.str = str;  
}  
  
public String toString() {  
return str;  
}  
}  
  
import java.util.HashSet;  
  
public class CollClient {  
  
public static void main(String ... sss) {  
HashSet myMap = new HashSet();  
String s1 = new String("das");  
String s2 = new String("das");  
NameBean s3 = new NameBean("abcdef");  
NameBean s4 = new NameBean("abcdef");  
  
myMap.add(s1);  
myMap.add(s2);  
myMap.add(s3);  
myMap.add(s4);  
}
```

```
        System.out.println(myMap);  
    }  
}
```

- 1.das abcdef abcdef
- 2.das das abcdef abcdef
- 3.das abcdef
- 4.None Of the above

Explanation :

A is the correct answer.

you have to implements 'equals' and 'hashCode' methods to get unique Set for user defined objects.

Question - 26

When a byte is added to a char, what is the type of the result?

- 1.byte
- 2.int
- 3.long
- 4.non of the above

Explanation :

B is the correct answer.

The result of all arithmetic performed with the binary operators (not the assignment operators) is an int, a long, a float, or a double. Here byte and char are promoted to int, so the result is an int.

Question - 27

What is the output of the below code ?

```
class A extends Thread {  
    public void run() {  
        System.out.print("A");  
    }  
}  
class B {  
    public static void main (String[] args) {  
        A a = new A();  
        a.start();  
        a.start(); // 1  
    }  
}
```

- 1.compile time error

- 2.Runtime Exception
- 3.the code compile and runs fine
- 4.none of the above

Explanation :

B is the correct answer.

If the start method is invoked on a thread that is already running, then an `IllegalThreadStateException` will probably be thrown

Question - 28

What is the output of the below code ?

```
class C{
    public static void main(String a[])    {
        int i1=9;
        int i2;
        if(i1>3) {
            i2=8;
        }
        System.out.println(i2);
    }
}
```

- 1.compile time error
- 2.Runtime error
- 3.prints 8
- 4.prints 0

Explanation :

A is the correct answer.

since variable i1 is not final the value is not known at compiletime itself.so generate compile time error

Question - 29

Which of the following classes will not allow unsynchronized read operations by multiple threads?

- 1.Vector
- 2.TreeMap
- 3.TreeSet
- 4.HashMap

Explanation :

A is the correct answer.

All the methods of Vector is synchronized.

Question - 30

Which of the following can be used to define a constructor for this class, given the following code:

```
public class Test {  
    ...  
}
```

1. public void Test() {...}
2. public Test() {...}
3. public static Test() {...}
4. public static void Test() {...}

Explanation :

A is the correct answer.

Constructor should not have any return type and should not be static.

Question - 31

What is the output for the bellow code ?

```
public class Bean {  
    private String str;  
  
    Bean(String str ){  
        this.str = str;  
    }  
  
}  
  
public class Test {  
  
    public static void main(String argv[]){  
  
        TreeSet myMap = new TreeSet();  
        String s1 = new String("abcdef");  
        String s2 = new String("abcdef");  
        Bean s3 = new Bean("abcdef");  
        Bean s4 = new Bean("abcdef");  
  
        myMap.add(s1);  
        myMap.add(s2);  
        myMap.add(s3);  
        myMap.add(s4);  
  
        System.out.println(myMap);  
  
    }  
}
```

}

- 1.abcdef abcdef abcdef abcdef
- 2.abcdef abcdef abcdef
- 3.java.lang.ClassCastException
- 4.None of the above

Explanation :

C is the correct answer.

Every Object need to implements comparable interface to sort the objects in TreeSet .

Question - 32

What is the output of the below code ?

```
class base
{
    base(int c)
    {
        System.out.println("base");
    }
}
class Super extends base
{
    Super()
    {
        System.out.println("super");
    }
}
public static void main(String [] a)
{
    base b1=new Super();
}
}
```

- 1.compile time error
- 2.runtime exceptione
- 3.the code compiles and runs fine
- 4.None of the above

Explanation :

A is the correct answer.

If super class has different constructor other then default then in the sub class you can't use default constructor.

Question - 33

What is the output of the below code ?

```

class C {
public static void main(String[] args) {
    char c1=65;
    switch(c1){
        case 'A':
            System.out.println("one");
        default:
            System.out.println("two");
        case 'b':
            System.out.println("three");
    }
}
}

```

- 1.prints one twot hree
- 2.prints two three
- 3.compile time error
- 4.Runtime exception

Explanation :

A is the correct answer.

char is a legal value for switch clause

Question - 34

What is the output of the below code ?

```

import java.util.NavigableMap;
import java.util.concurrent.ConcurrentSkipListMap;

public class Test {
    public static void main(String... args) {

        NavigableMap <Integer, String>navMap = new
        ConcurrentSkipListMap<Integer, String>();

        navMap.put(4, "April");
        navMap.put(5, "May");
        navMap.put(6, "June");
        navMap.put(1, "January");
        navMap.put(2, "February");
        navMap.put(3, "March");

        System.out.print(navMap.firstEntry());

    }
}

```

- 1.Compile error : No method name like firstEntry()
- 2.1=January

3.6=June

4.4=April

Explanation :

B is the correct answer.

u firstEntry() Returns a key-value mapping associated with the least key in this map, or null if the map is empty.

Question - 35

What is the output of the below code ?

```
public class C {  
    public C(){  
    }  
}  
  
public class D extends C {  
    public D(int i){  
        System.out.println("tt");  
    }  
    public D(int i, int j){  
        System.out.println("tt");  
    }  
}  
  
public Test{  
    public static void main(String[] args){  
        C c = new D();  
    }  
}
```

- 1.It compile without any error
- 2.It compile with error
- 3.Can't say
- 4.None of the above

Explanation :

B is the correct answer.

C c = new D(); NOT COMPILE because D don't have default constructor. If super class has different constructor other then default then in the sub class you can't use default constructor

Question - 36

What is the output of the below code ?

```
import java.util.ArrayList;
import java.util.List;
import java.util.NavigableSet;
import java.util.TreeSet;

public class Test {
    public static void main(String... args) {

        List<Integer> lst = new ArrayList<Integer>();
        lst.add(34);
        lst.add(6);
        lst.add(2);
        lst.add(8);
        lst.add(7);
        lst.add(10);

        NavigableSet<Integer> nvset = new TreeSet(lst);
        System.out.println(nvset.lower(6)+" "+nvset.higher(6)+ "
"+ nvset.lower(2));

    }
}
```

1.1 2 7 10 34 null

2.2 7 null

3.2 7 34

4.1 2 7 10 34

Explanation :

B is the correct answer.

☐ lower() Returns the greatest element in this set strictly less than the given element, or null if there is no such element.

☐ higher() Returns the least element in this set strictly greater than the given element, or null if there is no such element.

Question - 37

What is the output of the below code ?

```
public class SuperClass {
    private int doIt(){
        System.out.println("super doIt()");
        return 1;
    }
}

public class SubClass extends SuperClass{

    public long doIt()
    {
        System.out.println("subclas doIt()");
        return 0;
    }
}
```

```

    public static void main(String... args)
    {
        SuperClass sb = new SubClass();

        sb.doIt();

    }
}

```

- 1.subclas doIt()
- 2.super doIt()
- 3.Compile with error
- 4.None of the above

Explanation :

C is the correct answer.

The method doIt() from the type SuperClass is not visible.

Question - 38

What is the output of the below code ?

```

class bike
{
}
class arr extends bike{
public static void main(String[] args) {
arr[] a1=new arr[2];
bike[] a2;
a2=a1;                //3
arr[] a3;
a3=a1;                //5
}}

```

- 1.compile time error at line 3
- 2.compile time error at line 5
- 3.Runtime exception
- 4.The code runs fine

Explanation :

D is the correct answer.

bike is the superclass of arr.so they are compatible(superobject=subobject) but subobject=superobject not allowed

Question - 39

Which one of the following is a limitation of subclassing the Thread class?

- 1.You must catch the ThreadDeath exception.
- 2.You must implement the Threadable interface.
- 3.You cannot have any static methods in the class
- 4.You cannot subclass any other class.

Explanation :

D is the correct answer.

Java don't support multiple inheritance.Subclassing the Thread Class , you can't extend any other class.

Question - 40

What is the output of the bellow code ?

```
public class Test {  
  
    public static void main(String[] args) {  
        Integer in = new Integer(null);  
        System.out.println(in.intValue());  
    }  
  
}
```

- 1.0
- 2.Compile with error because , Integer in = new Integer(null);
- 3.java.lang.NumberFormatException
- 4.None of the above

Explanation :

C is the correct answer.

An Integer expression can have a null value. If your program tries to autounbox null, it will throw a NullPointerException

Question - 41

What will be the result of compiling the following code:

```
public class Test {  
    public static void main(String... args) {  
        flipFlop("hello", new Integer(4), 2004);  
    }  
}
```

```

private static void flipFlop(String str, int i, Integer iRef) {
    System.out.println(str + " (String, int, Integer)");
}

private static void flipFlop(String str, int i, int j) {
    System.out.println(str + " (String, int, int)");
}

}

```

- 1.the code will compile and print hello (String, int, Integer)
- 2.the code will compile an print hello (String, int, int)
- 3.Complile time error.
- 4.None of the above

Explanation :

C is the correct answer.

The method flipFlop(String, int, Integer) is ambiguous. if you use flipFlop("hello", 10, 2004); then it will not ambiguous.

Question - 42

What is the output of the bellow code ?

```

class C1
{
    static interface I
    {
        static class C2
        {
        }
    }
}

public static void main(String a[])
{
    C1.I.C2 ob1=new C1.I.C2();
    System.out.println("object created");
}
}

```

- 1.prints object created
- 2.Compile time error
- 3.Runtime Excepion
- 4.None of the above

Explanation :

A is the correct answer.

A static interface or class can contain static members.Static members can be accessed without instantiating the particular class

Question - 43

Each element must be unique

Duplicate elements must not replace old elements.

Elements are not key/value pairs.

Accessing an element can be almost as fast as performing a similar operation on an array.

Which of these classes provide the specified features?

1. LinkedList
2. TreeMap
3. HashMap
4. HashSet

Explanation :

D is the correct answer.

element must be unique in Set

Question - 44

What is the output of the below code?

```
import java.util.LinkedList;
import java.util.Queue;

public class Test {
    public static void main(String... args) {

        Queue<String> q = new LinkedList<String>();
        q.add("newyork");
        q.add("ca");
        q.add("texas");
        show(q);

        public static void show(Queue q) {
            q.add(new Integer(11));
            while (!q.isEmpty())
                System.out.print(q.poll() + " ");
        }
    }
}
```

1. Compile error : Integer can't add
2. newyork ca texas 11
3. newyork ca texas

4. None of the above

Explanation :

B is the correct answer.

□ q was originally declared as Queue<String>, But in show() method it is passed as an untyped Queue. nothing in the compiler or JVM prevents us from adding an Integer after that.

W If the show method signature is public static void show(Queue<String> q) then you can't add Integer, Only String allowed. But public static void show(Queue q) is untyped Queue so you can add Integer.

z poll() Retrieves and removes the head of this queue, or returns null if this queue is empty.

Question - 45

What is the output of the below code?

```
class A extends Thread {  
    private int i;  
    public void run() {  
        i = 1;  
    }  
    public static void main(String[] args) {  
        A a = new A();  
        a.run();  
        System.out.print(a.i);  
    }  
}
```

1. Prints nothing
2. Prints: 1
3. Prints: 01
4. Compile-time error

Explanation :

B is the correct answer.

a.run() method was called instead of a.start(); so the full program runs as a single thread so a.run() is guaranteed to complete

Question - 46

The _____ exception that sleep throws when another thread interrupts the current thread while sleep is active ?

1. InterruptedException
2. IOException
3. SleepException
4. None of the above

Explanation :

A is the correct answer.

InterruptedException that sleep throws when another thread interrupts the current thread while sleep is active

Question - 47

What can cause a thread to stop executing?

- 1) The program exits via a call to `System.exit(0);`
- 2) Another thread is given a higher priority
- 3) A call to the thread's stop method.
- 4) A call to the halt method of the `Thread` class?

1.1 , 2 AND 3

2.1 , 2 AND 4

3.1 AND 4

4. None of the above

Explanation :

A is the correct answer.

can cause a thread to stop executing, not what will cause a thread to stop executing. Java threads are somewhat platform dependent and you should be careful when making assumptions about Thread priorities

Question - 48

What is the output of the below code?

```
class H {  
    public static void main (String[] args) {  
        String s1 = "HHH";  
        StringBuffer sb2 = new StringBuffer(s1);  
        System.out.print(sb2.equals(s1) + "," + s1.equals(sb2));  
    }  
}
```

1. Prints: false,false

2. Prints: true,false

3. Prints: false,true

4. Prints: true,true

Explanation :

A is the correct answer.

s1 and sb2 are pointing to different object references

Question - 49

What will be the result of compiling the following code:


```

public class Test {
    public static void main (String args []) {
        int age;
        age = age + 1;
        System.out.println("The age is " + age);
    }
}

```

- 1.the code will compile an print 1.
- 2.the code will compile an print 0.
- 3.Complile time error.
- 4.None of the above

Explanation :

C is the correct answer.

local variable should be initialized.

Question - 50

What is the output of the below code?

```

public class A {
    static{System.out.println("static");}
    { System.out.println("block");}
    public A(){
        System.out.println("A");
    }
}

public static void main(String[] args){
    A a = new A();
}

```

- 1.A block static
- 2.static block A
- 3.static A
- 4.A

Explanation :

B is the correct answer.

First execute static block, then block then constructor.

Question - 51

What is the output of the below code?

```

public class A implements Serializable {
    transient int a = 7;
    static int b = 9;
}

```

```

    }

    public class B implements Serializable {

        public static void main(String... args){
            A a = new A();
            try {
                ObjectOutputStream os = new ObjectOutputStream(
                    new FileOutputStream("test.ser"));
                os.writeObject(a);
                os.close();
                System.out.print( ++ a.b + " ");

                ObjectInputStream is = new ObjectInputStream(new
FileInputStream("test.ser"));
                A s2 = (A)is.readObject();
                is.close();
                System.out.println(s2.a + " " + s2.b);
            } catch (Exception x)
            {
                x.printStackTrace();
            }

        }

    }
}

```

1.9 0 9

2.9 7 9

3.Runtime Exception

4.Compile with error

Explanation :

A is the correct answer.

static and transient variables are not serialized when an object is serialized.

Question - 52

What is the output of the below code?

```

class C{
    static int s;
    public static void main(String a[]){
        C obj=new C();
        obj.m1();
        System.out.println(s);
    }
    void m1();
    {
        int x=1;
        m2(x);
        System.out.println(x+"");
    }
    void m2(int x){
        x=x*2;
    }
}

```

```
s=x;  
}}
```

- 1.prints 1,2
- 2.prints 2,0
- 3.prints 2,2
- 4.compile time error

Explanation :

A is the correct answer.

Only objects and arrays are passed by reference.other are passed by value.s is a static variable which is global to the class

Question - 53

What will happen when you attempt to compile and run the following code?

```
public class Inc{  
public static void main(String argv[]){  
    Inc inc = new Inc();  
    int i =0;  
    inc.fermin(i);  
    i = i++;  
    System.out.println(i);  
}  
void fermin(int i){  
    i++;  
}  
}
```

- 1.Compile time error
- 2.Output of 2
- 3.Output of 1
- 4.Output of 0

Explanation :

D is the correct answer.

The method fermin only receives a copy of the variable i and any modifications to it are not reflected in the version in the calling method. The post increment operator ++ effectively modifies the value of i after the initial value has been assigned to the left hand side of the equals operator. This can be a very tricky concept to understand

Question - 54

What is the output ?

```
public class Test {
```

```

public static void main(String... args) {
    int arr[] = {2,4,8,16,32};
    int result = 0;
    for (int i=0; i<arr.length; i++){
        result = result + arr[i];
    }
    System.out.println("Output old: " + result);
    result = 0;
    for(int a:arr)
    {
        result = result + a;
    }
    System.out.println("Output new: "+ result);
}
}

```

1. Output old: 62 Output new: 62
2. Output old: 62 Output new: 0
3. Compile error
4. None of the above

Explanation :

A is the correct answer.

for(int a:arr) is "for each int a in arr." in jdk 1.5

Question - 55

Thread class `setPriority(int newPriority)` method allow _____
MAX_PRIORITY ?

1. 10
2. 11
3. 5
4. 12

Explanation :

A is the correct answer.

Thread has `MAX_PRIORITY = 10` and `MIN_PRIORITY = 1` and priority should not be `newPriority > MAX_PRIORITY || newPriority < MIN_PRIORITY`

Question - 56

What will happen when you attempt to compile and run the following code

```

public class Test extends Thread{

```

```

public static void main(String argv[]){
    Test b = new Test();
    b.start();
}
public void run(){
    System.out.println("Running");
}
}

```

- 1.Compilation and run but no output
- 2.Compilation and run with the output "Running"
- 3.Compile time error with complaint of no Thread target
- 4.Compile time error with complaint of no access to Thread package

Explanation :

B is the correct answer.

This is perfectly legitimate if useless sample of creating an instnace of a Thread and causing its run method to execute via a call to the start method. The Thread class is part of the core java.lang package and does not need any explicit import statement.

Question - 57

What is the output of the bellow code ?

```

public enum Test {
    BREAKFAST(7, 30), LUNCH(12, 15), DINNER(19, 45);

    private int hh;

    private int mm;

    Test(int hh, int mm) {
        assert (hh >= 0 && hh <= 23) : "Illegal hour.";
        assert (mm >= 0 && mm <= 59) : "Illegal mins.";
        this.hh = hh;
        this.mm = mm;
    }

    public int getHour() {
        return hh;
    }

    public int getMins() {
        return mm;
    }

    public static void main(String args[]){
        Test t = BREAKFAST;
        System.out.println(t.getHour() + ":" + t.getMins());
    }
}

```

1.7:30

2.Compile With Error

- 3.Compile Without Error but Runtime Exception
- 4.None of the above

Explanation :

B is the correct answer.

All the enum static like BREAKFAST treat as constructor. Enum constructors Each constant declaration can be followed by an argument list that is passed to the constructor of the enum type having the matching parameter signature. An implicit standard constructor is created if no constructors are provided for the enum type. As an enum cannot be instantiated using the new operator, the constructors cannot be called explicitly. Output is : 7:30

Question - 58

What is the output ?

```
public class Test {
    public static void main(String... args) {
        printUs("ONE", "TWO", "THREE");
        printUs("FOUR", "FIVE");
        printUs(new String[]{"SIX", "SEVEN"});
        printUs();
    }

    private static void printUs(String... args) {
        System.out.println("Var args method");
        for (String s : args) {
            System.out.println(s);
        }
    }
}
```

- 1.Var args method ONE TWO THREE Var args method FOUR FIVE Var args method SIX SEVEN Var args method
- 2.Var args method ONE TWO THREE Var args method FOUR FIVE Var args method SIX SEVEN
- 3.Var args method SIX SEVEN
- 4.None of the above

Explanation :

A is the correct answer.

The three periods after the final parameter's type indicate that the final argument may be passed as an array or as a sequence of arguments. Varargs can be used only in the final argument position. methods that can be called with variable-length argument list.

Question - 59

What is the output of the below code ?

```
public class A extends Integer{  
    public static void main(String[] args){  
        System.out.println("Hello");  
    }  
}
```

What is the output?

- 1.Hello
- 2.Compile Error
- 3.Runtime
- 4.None of the above

Explanation :

B is the correct answer.

final class can't be extended. Integer is final class.

Question - 60

What is displayed when the following piece of code is executed:

```
class Test extends Thread{  
    public void run() {  
        System.out.println("1");  
        yield();  
        System.out.println("2");  
        suspend();  
        System.out.println("3");  
        resume();  
        System.out.println("4");  
    }  
    public static void main(String []args){  
        Test t = new Test();  
        t.start();  
    }  
}
```

- 1.1 2 3 4
- 2.1 2 3
- 3.1 2
- 4.Nothing. This is not a valid way to create and start a thread.

Explanation :

C is the correct answer.

The code will run, but the thread suspends itself after displaying "2". Threads cannot un-suspend themselves (since they ARE suspended and therefore not running!).

